

The 'sopra-requirements' Package

Documentation for the 'sopra-requirements' package | Version v1.0.09

2. December 2019

Florian Sihler (florian.sihler@uni-ulm.de)

1 General

1.1 Why, what for?

This $\text{\LaTeX}$ package was written as part of the Sopra in the winter semester 2019 and summer semester 2020 and serves as a basis for the definition of requirements (functional, quality, ...) of *Team 20*. This documentation was created together with the `sopra-base.cls` as well as the `sopra-documentation.sty` package.

The `sopra-listings` package is used to visualize the individual code excerpts. The associated package should also be embedded in this document: .

1.2 Dependencies

This package includes the following packages:

- | | | |
|--|--------------------------|----------------------------|
| ▪ <code>tabu</code> ^(usetabu) | ▪ <code>longtable</code> | ▪ <code>etoolbox</code> |
| ▪ <code>booktabs</code> ^(notabu) | ▪ <code>xcolor</code> | |
| ▪ <code>colortbl</code> ^(usetabu) | ▪ <code>environ</code> | ▪ <code>fontawesome</code> |
| ▪ <code>makecell</code> | ▪ <code>amssymb</code> | ▪ <code>hyperref</code> |

All of these packages should be part of a common $\text{\LaTeX}$ distribution.

1.3 Installation

The package is not delivered as a `.dtx`, which is why the following options arise:

- The package can be placed in the same directory as the document. In this case the inclusion instruction reads:
`\usepackage{sopra-requirements}`
- The package can be placed in a subdirectory/a directory shipped with the document. In this case it is specified via the (relative) path:
`\usepackage{./My/Path/to/sopra-requirements}`
- The package can be placed (via a symlink or similar) in one's own `texmf` tree. For example, on Linux using `texlive`, the package can be placed here: `~/texmf/tex/latex/`. The directory can be created and then updated using `texhash ~/texmf`. Now the package can be used like any other installed package:
`\usepackage{sopra-requirements}`

1.4 Further peculiarities

In version v1.0.09 (`\thesorversion`^{→ p. 3}) there are no further peculiarities.

2 Package Configuration

2.1 Accepted Parameters

- ▷ `usetabu` (`notabu`)

By default, for visual reasons, this package uses `tabu` to visualize the tables. Should the (bug) fixes contained in this package (no longer) work, or should one prefer the `booktabs` tables used in `notabu`, this option can be changed accordingly.

- ▷ `intoc` (`notoc`)

By default, the `tabu` package includes the requirements in the table of contents (as a subsubsection, so by default it is only shown in the TOC of the PDF viewer). If this is not desired, it can be disabled with `notoc`.

- ▷ `showtag` (`noshowtag`)

Shows, in addition to the ID, the (internal) identifier of the requirement. This is merely a visual option that can be freely modified as desired.

- ▷ `noshowallfields` (`showallfields`)

If `showallfields` is set, fields in `env@requirement`^{→ p. 2} are also shown even if they have not been given a value.

2.2 Colors

With `usetabu`, the color `HeaderColor` defines the color of the title column and `NextHeaderColor` the color in the case of a page break. The colors can be changed like any other. Within `sopra-base`, a color adapted to the design is automatically chosen.

3 Commands and Environments

Note that the prefix `env@` only indicates the nature of an environment and is not part of the actual identifier!

Also: For all titles and lengths to be resolved correctly, the document usually has to be compiled twice to produce a correct display.

3.1 Defining Requirements

- ▷ `env@requirements[Start-ID]{functional/quality}`

Starts a new block of requirements, within which `env@requirement`^{→ p. 2} is available. By default, the requirements are numbered consecutively from 1 for the whole document; however, should this starting value need to be changed (which, apart from testing purposes, should not actually make sense, since this can cause some requirements to exist twice!), this is possible by setting `Start-ID`. It must be specified whether these are *functional* (functional) or *non-functional* (quality) requirements. All requirements nested within this are counted as such. The two variants have their own counters!. Developer note: The required values are stored in `\@sor@fnctl` and `\@sor@qltl`. The prefix accordingly in `\sor@functional@requirementprefix` and `\sor@quality@requirementprefix`.

- ▷ `env@requirement[ID]{identifier}`

This environment is only available within `env@requirements` → p. 2! This can be used to define a single requirement, where the ID results either sensibly by incrementing the counter or by explicit specification. An ID must always be numeric. Furthermore, an identifier can be specified here, which by default represents the title of the requirement, as long as it is not explicitly changed using `\setTitle` → p. 3. An identifier must not contain umlauts or other special characters. Formatting and other $\text{\LaTeX 2}_{\epsilon}/\text{\TeX}$ commands cannot be used here either. The `{identifier}` can also be used to reference this requirement using `\preqref` → p. 3. There must not be a duplicate identifier.

3.2 Setting Data in a Requirement

The following commands can only be used within `env@requirement` → p. 2! It remains to work on a mechanism that allows one to define custom fields. It is planned to use `translate` in order to at least be able to translate the data fields, or in general define them freely!

▷ `\setDescription{description text}`

Sets an explanation for the respective requirement. In principle anything can be set here, from an `env@eagle-map` to `env@itemize` to completely normal $\text{\LaTeX}$.

▷ `\setJustification{justification text}`

Sets the justification for the respective requirement. The same applies with regard to the content as for `\setDescription` → p. 3.

▷ `\setDependencies{comma-separated list}`

Allows listing all dependencies using schemas accepted by `\preqref` → p. 3. For example: `\setDependencies{FA1,QA42,main-menu}`.

▷ `\setTitle{title}`

Allows changing the title of the requirement. This does not change the identifier set in `env@requirement` → p. 2! (This can be made clear by `showtag`).

▷ `\setPriority{number}`

Allows defining the priority of the respective requirement. The number must be between (inclusive) 0 and 5.

3.3 General Commands

▷ `\thesorversion`

Returns the current version of the package. So `\thesorversion` results in: `v1.0.09`

Note: via `\value{sorversion}` the version can be obtained as a 4-digit number: `1009`.

▷ `\iskip`

Sets a *negative spacing*, which can be used when a spacing has been set too large. This will only be necessary when introducing custom environments or similar, and is intended here merely as a kind of inspiration.

▷ `\preqref[Text]{requirement marker}`

This command sets a reference to a requirement set using `env@requirement`^{→ p. 2}. Two valid markers are generated for each:

- The identifier can be used.
- The abbreviation for the requirement type plus the respective number can be used (FA13, QA9, ...)

This command can also be used with an asterisk. In this case the identifier of the requirement is not output, only the abbreviation (for example FA42) together with the page on which the requirement is defined. If `Text` is set, it is displayed instead of the requirement name.

▷ `\pref{hyperlink marker}{text}`

Used by `\preqref`^{→ p. 3} and simply outputs a link with a page number. Can be modified (/overridden) to adapt the behavior of `\preqref`^{→ p. 3}.

▷ `\sorSetPriority{number}`

Works analogously to `\setPriority`^{→ p. 3}, but can also be used outside of `env@requirement`^{→ p. 2}.

3.4 Examples

Let us define two requirements in the following:

```
1 \begin{requirements}{functional}
2   \begin{requirement}{a-duck}
3     \setTitle{Duck}
4     \setDescription{There must be ducks. So that they stay alive, there is \preqref{
5       feed-ducks}}
6     \setJustification{Because they are cute}
7   \end{requirement}
8   \begin{requirement}{feed-ducks}
9     \setTitle{Feed duck}
10    \setDescription{It must be possible to feed the ducks from \pref{FA1}. We have
11      : \preqref[Ducks]{FA1}.}
12    \setJustification{Ducks must (be allowed to) live!!}
13    \setDependencies{a-duck}
14    \setPriority{4}
15  \end{requirement}
16 \end{requirements}
```

The result is (with tabu) the following:

ID	FA1	a-duck
TITLE:	Duck	
DESCRIPTION:	There must be ducks. So that they stay alive, there is FA2: 'Feed duck' → p. 4	
JUSTIFICATION:	Because they are cute	

ID	FA2	feed-ducks
TITLE:	Feed duck	★★★★☆
DESCRIPTION:	It must be possible to feed the ducks from FA1: 'Duck' $\rightarrow p.4$. We have: Ducks (FA1) $\rightarrow p.4$.	
JUSTIFICATION:	Ducks must (be allowed to) live!!	
DEPENDENCIES:	a-duck $\rightarrow p.4$	